B M S COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(Autonomous college under VTU)
P.O. Box No.: 1908, Bull Temple Road, Bangalore-560 019
2026-2027



Alternate Assessment Report on Network Programming

"Performance Analysis of AODV Routing Protocol in a MANET Using NS-3"

with a Comparison against OLSR and DSDV

Course Title: Network Programming | Course Code: 23CS7PCNWP

Submitted by

Tanmay Agarwal (1WA23CS010)
Viamrsh Buyanni (1WA23CS039)
Yeruva Taniya Paul (1WA23CS053)
Vijeth MD (1WA23CS041)

Work carried out at

Department of CSE, BMSCE

Faculty in charge

Prof. Sunayana S

Assistant Professor, Department of Computer Science and Engineering

Date of Submission: 3 November 2026

INDEX

Sl. No.	Chapter Title	Page No.
1	Abstract	1
2	Objectives	2
3	Introduction	3
4	Literature Review	5
5	Research Gap	7
6	Problem Statement	8
7	Methodology and Implementation	9
8	Performance Evaluation	15
9	Output and Results	16
10	Conclusion	21
11	References	22

1. Abstract

This report studies the AODV routing protocol in a mobile ad hoc network (MANET), where wireless nodes forward packets for one another without infrastructure, and compares it with the proactive protocols OLSR and DSDV. The network was implemented in C++ with the ns-3.48 simulator: nodes move under the Random Waypoint model in a 700 m × 700 m area and use IEEE 802.11b ad-hoc Wi-Fi with a 250 m range. A UDP client–server application built on the ns-3 socket API sends constant bit rate traffic, and FlowMonitor and packet traces measure the packet delivery ratio (PDR), throughput, delay, jitter and routing overhead. The number of nodes, node speed and packet rate were varied one at a time, with ten random topologies per setting and protocol (420 runs). AODV delivered the most packets, 77.9% on average against 66.2% for OLSR and 65.0% for DSDV, and significantly more in 12 of 13 settings than OLSR and 9 of 13 than DSDV. From 1 to 15 m/s, AODV kept its PDR near 75% while OLSR and DSDV fell to 50% and 47%. The cost was about twice as many routing control packets and a higher delay.

Keywords: MANET, AODV, OLSR, DSDV, ns-3, socket programming, routing overhead, packet delivery ratio.

2. Objectives

2.1 Primary Objective

To implement a mobile ad hoc network in ns-3 in which a UDP client–server application exchanges data over multi-hop routes, to analyse how AODV performs when the number of nodes, the node speed and the traffic load are changed, and to compare it with the proactive protocols OLSR and DSDV.

2.2 Specific Objectives

1. To understand socket programming concepts: how a UDP client and server exchange datagrams through calls such as `socket()`, `bind()`, `connect()`, `send()` and `recvfrom()`.
2. To implement UDP client–server communication between two mobile nodes in ns-3, with a constant bit rate (CBR) source on node 0 and a sink on node $N-1$.
3. To study how AODV discovers and repairs multi-hop routes with Route Request, Route Reply and Route Error messages sent over UDP port 654.
4. To configure an IEEE 802.11b ad-hoc network (11 Mbps, 250 m range) with Random Waypoint mobility in a 700 m × 700 m area.
5. To measure the packet delivery ratio, throughput, delay, jitter and packet loss of the data flow, and the routing overhead of each protocol.
6. To analyse network performance while varying the number of nodes, the node speed and the packet rate, and to compare AODV with OLSR and DSDV on identical topologies, repeating every setting ten times.
7. To validate the measurements with analytical checks and to report means, standard deviations, 95% confidence intervals and significance tests.

2.3 Expected Outcomes

A reproducible ns-3 program and scripts that run all 420 simulations, CSV files with one row per run, graphs comparing the three protocols, and an explanation of the observed behaviour in terms of each protocol's mechanisms, together with the limitations of the simulated scenario.

3. Introduction

3.1 Network Programming and the Application

Network programming is the writing of programs that exchange data with programs on other machines. The operating system provides the protocol stack (Wi-Fi or Ethernet, IP, TCP and UDP), and applications reach it through system calls, most commonly the Berkeley (BSD) sockets interface [1]. A socket is a communication endpoint identified by an IP address and a port number: the application writes data into it and the stack delivers the data to the peer socket [2].

The application studied here is multi-hop data delivery in a mobile ad hoc network (MANET). A UDP client on one mobile node sends a stream of packets to a UDP server on another node. There is no access point or wired router: the other mobile nodes forward the packets, and the AODV routing protocol, which itself communicates through UDP sockets, decides which nodes forward them [3, 4].

3.2 Socket Programming, Transport Protocols and System Calls

A UDP application uses a short sequence of system calls. The server calls `socket()` to create a datagram socket, `bind()` to attach it to a known port and `recvfrom()` to receive datagrams. The client calls `socket()`, optionally `connect()` to fix the destination, and `send()` or `sendto()` for every datagram; both call `close()` at the end [1, 5]. TCP adds connection set-up with `listen()`, `accept()` and `connect()`, and provides reliable, ordered delivery with retransmission and congestion control. UDP provides none of these; it only adds an 8-byte header with ports, length and checksum to each datagram [6].

UDP is the right transport for this study for two reasons. First, a constant bit rate stream over UDP is neither slowed down nor repeated by the transport layer, so every lost or delayed packet directly reflects the routing protocol and the wireless channel; TCP would retransmit lost segments and reduce its rate, hiding exactly the effects being measured. Second, AODV itself sends its control messages as UDP datagrams to port 654 [4]. ns-3 provides a socket API modelled on BSD sockets [7], and Section 7.5 shows how the client, the server and AODV use it.

3.3 Reactive Routing and AODV

MANET routing protocols are commonly divided into proactive (table-driven) protocols, such as DSDV [8] and OLSR [9], which keep routes to every destination up to date with periodic control messages, and reactive (on-demand) protocols, such as DSR [10] and AODV [11], which look for a route only when a source has data to send. Reactive protocols avoid maintaining unused routes, at the cost of a discovery delay before the first packets of a flow and after every route break. In ns-3, OLSR sends HELLO messages every 2 s and topology control (TC) messages every 5 s, and DSDV sends full routing updates every 15 s plus triggered updates when routes change.

In AODV, a source without a route broadcasts a Route Request (RREQ). Each node that receives it records a reverse route to the source and rebroadcasts it. The destination, or a node with a fresh enough route, returns a Route Reply (RREP) along the reverse path, which installs the forward route

hop by hop. When a link on an active route breaks, a Route Error (RERR) informs the nodes that use it, and the source starts a new discovery. Destination sequence numbers keep routes loop-free and distinguish fresh routes from stale ones [4].

3.4 Tools and Languages

The simulation is written in C++ for the ns-3 discrete-event network simulator, release 3.48 [12, 13], following the ns-3 tutorial [14]. ns-3 supplies the Wi-Fi, mobility, Internet stack, AODV and FlowMonitor models [15], and the program is compiled with ns-3's CMake-based build system and Ninja. Bash scripts run single scenarios and the complete set of experiments; Python 3 with pandas and SciPy processes the CSV results and runs the statistical tests, and matplotlib draws the graphs. The work was done on macOS on Apple Silicon, and the code and results are kept in a Git repository on GitHub [16].

3.5 Motivation

AODV's performance depends on the network around it: a path may not exist in a sparse network, routes break often when nodes move fast, and data and control packets compete for the channel under heavy load. Studying these effects with real devices would need many nodes and repeatable movement. A simulator allows the same scenario to be repeated exactly, one parameter to be changed at a time and every packet to be counted. This project uses that ability to measure AODV under controlled conditions, to compare it fairly with OLSR and DSDV on exactly the same topologies, and to report how much the results vary between random topologies.

4. Literature Review

4.1 MANET Routing Protocols

Abolhasan, Wysocki and Dutkiewicz reviewed MANET routing protocols, grouped them into proactive, reactive and hybrid families, and compared their control overhead and scalability [17]. Perkins and Bhagwat proposed DSDV, which adapts distance-vector routing to mobile nodes by tagging each route with a sequence number issued by the destination, so that nodes prefer fresh routes and avoid loops [8]. Johnson and Maltz proposed DSR, a reactive protocol that discovers routes on demand and carries the complete route in each packet header [10]. Perkins and Royer introduced AODV, which combines DSDV's sequence numbers with on-demand discovery and keeps only hop-by-hop next-hop entries for routes in active use [11]; it was later standardised as RFC 3561 [4].

4.2 Comparative Performance Studies

Broch *et al.* compared DSDV, TORA, DSR and AODV in a packet-level simulation with Random Waypoint mobility and showed that the on-demand protocols kept high delivery ratios at high mobility while DSDV degraded [18]. Johansson *et al.* compared AODV, DSR and DSDV in realistic scenarios such as a conference, event coverage and a disaster area, and found that the relative performance of the protocols depends on the scenario [19]. Perkins, Royer, Das and Marina compared AODV and DSR under varying load, mobility and network size, and found that DSR performed better in less stressful conditions while AODV did better as load and mobility increased [20]. Marina and Das extended AODV to AOMDV, which finds several loop-free, link-disjoint paths in one discovery so that a broken route can be replaced without a new flood [21].

4.3 Mobility and Radio Models

Camp, Boleng and Davies surveyed mobility models for ad hoc network research and showed that the chosen model can change the measured performance of a protocol [22]. Yoon, Liu and Noble showed that the average speed under Random Waypoint decays over time when speeds are drawn from a range that starts at zero [23]; in this project every node moves at one constant speed, so this effect does not arise. Kotz *et al.* tested common simulation assumptions, such as a circular radio range with perfect reception inside it, against measurements from a large outdoor experiment, and showed that they can make simulated results differ substantially from real ones [24].

4.4 Credibility of Simulation Studies and Tools

Kurkowski, Camp and Colagrosso surveyed the simulation studies published at ACM MobiHoc from 2000 to 2005 and found that many did not report enough detail, such as the simulator settings or the number of runs, to be repeated, and they listed common pitfalls [25]. Andel and Yasinsac similarly questioned the credibility of MANET simulations and recommended documenting all parameters and reporting statistically sound results [26]. Riley and Henderson described the design of ns-3, whose C++ models follow real implementations closely and expose a BSD-like socket interface

[12], and Carneiro, Fortuna and Ricardo proposed FlowMonitor, which records per-flow packet counts and delays in ns-3 [15].

4.5 Recent Work on AODV

Mammeri reviewed and classified reinforcement-learning-based routing protocols, more than sixty of which have been proposed since the mid-1990s [27]. Alameri *et al.* carried out a systematic review of modifications to AODV's route discovery, noting that AODV needs few resources because its control messages and routing tables are small [28]. Quy *et al.* surveyed routing algorithms for networks that combine MANETs with the Internet of Things [29]. Santana *et al.* used bio-inspired multi-objective algorithms to tune AODV's route recovery and reported lower routing delay, packet loss and route load than standard AODV [30]. Njoroge, Ndia and Mwangi reviewed AODV extensions for performance improvement and found that most modify route discovery, selection or maintenance using quality-of-service parameters, machine learning or signal strength [31].

4.6 Summary

The literature shows that MANET routing performance depends strongly on density, mobility, load and the models used, that many published simulations are hard to repeat, and that recent AODV improvements are all judged against standard AODV. This motivates a carefully documented, repeated-run evaluation of standard AODV.

5. Research Gap

5.1 Research Gaps Identified

The literature review points to the following gaps:

1. **Scenario dependence.** Protocol performance and even protocol rankings change with mobility, load and network size [18, 19, 20], yet many studies change several factors in one scenario, so the cause of a change cannot be identified.
2. **Credibility and repeatability.** Many published MANET simulations do not report the settings, number of runs or statistical confidence needed to repeat or trust them [25, 26].
3. **Single-topology results.** In a sparse network one random topology may connect the source and destination directly while another leaves them disconnected, so a result from one run, or a mean without its spread, can be misleading.
4. **Idealised models.** A circular radio range and synthetic mobility can make simulated results differ from real deployments [22, 23, 24].
5. **Reliable baselines.** The AODV variants in recent reviews are each compared with standard AODV in their own scenarios [28, 30, 31], so a documented, reproducible baseline for standard AODV is needed before an improvement can be judged.

5.2 Selected Research Gap

Gap 4 needs measured radio and movement traces and is left for future work. This project narrows down to gap 5, studied with the controls that gaps 1 and 3 call for. The selected research gap is: *there is no transparent, reproducible comparison of standard AODV with proactive baselines on a current ns-3 release that varies node density, node speed and traffic load one at a time, measures routing overhead, and reports how much the results vary between random topologies, with statistical confidence.*

5.3 Contribution

The project contributes a documented ns-3 program that runs AODV, OLSR or DSDV on identical topologies, scripts that run 420 simulations with fixed random streams, CSV files with one row per run, and an analysis that reports means, standard deviations, 95% confidence intervals and paired significance tests. It does not propose a new routing algorithm; it provides the reproducible baseline comparison that gaps 2 and 5 call for.

6. Problem Statement

6.1 Problem Statement

Addressing the research gap selected in Section 5.2, the problem is formulated as follows:

> *Design and implement, in ns-3.48, a mobile ad hoc network of N nodes over IEEE 802.11b ad-hoc Wi-Fi, in which a UDP client–server application sends constant bit rate traffic from node 0 to node N−1, and determine, using repeated runs with statistical confidence, how the delivery ratio, throughput, delay, jitter and routing overhead of AODV change when the number of nodes, the node speed and the packet rate are varied one at a time, and how AODV compares with OLSR and DSDV on the same topologies.*

6.2 Research Questions

1. How does node density (5 to 20 nodes in 700 m × 700 m) affect delivery and delay?
2. How does node speed (1 to 15 m/s) affect delivery and delay?
3. At what traffic load does the shared channel saturate, and how do throughput, delivery and delay respond?
4. How does AODV compare with OLSR and DSDV in delivery, delay and routing overhead, and which differences are statistically significant?

6.3 Scope and Assumptions

- All nodes are identical and use the ns-3 default settings of the protocol under test.
- The radio range is a hard 250 m cutoff, and there is one data flow, from node 0 to node $N-1$.
- Traffic runs from 2 s to 59 s of a 60 s simulation, so packets in flight at the end are not counted as lost.
- Data metrics cover the data flow only; routing control messages are counted separately as overhead.

7. Methodology and Implementation

7.1 Proposed Architecture

Figure 7.1 shows the architecture of the simulated system. Every node has the same protocol stack: an IEEE 802.11b physical layer and ad-hoc MAC, IPv4 with the routing protocol under test (AODV, OLSR or DSDV), UDP, and the ns-3 socket API. The source node runs the CBR client application and the destination runs the server; every other node acts only as a router. All nodes share one wireless channel. A data packet travels down the source's stack, through relays that forward it at the IPv4 layer using routes found by the routing protocol, and up the destination's stack. FlowMonitor observes the data flow at the IPv4 layer of the source and destination.

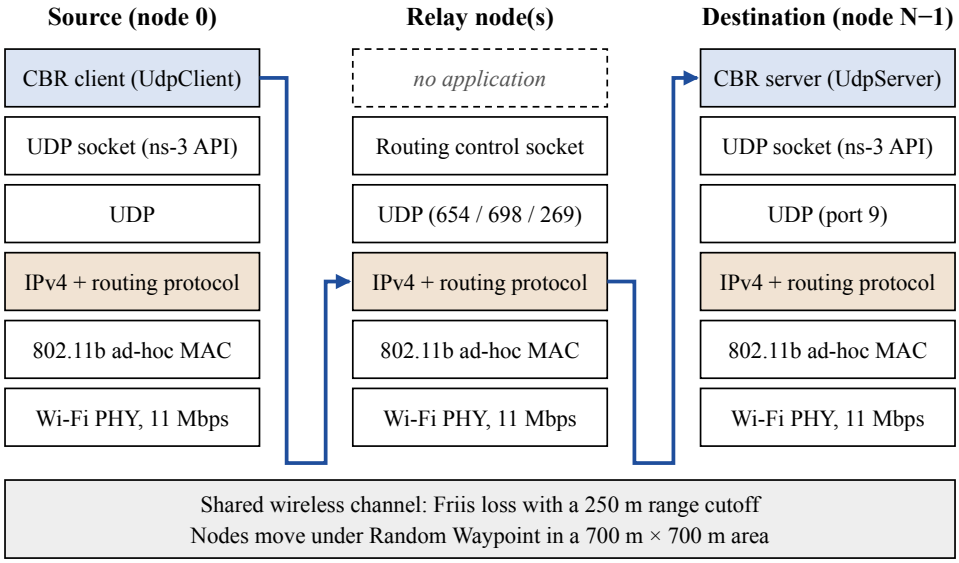


Figure 7.1: Proposed system architecture. Blue arrows show the path of a CBR data packet through one relay.

7.2 Network Communication Model

The network contains N nodes (10 by default) in a 700 m × 700 m square. All nodes use IEEE 802.11b [32] in ad-hoc mode, with the data rate fixed at 11 Mbps for both unicast and broadcast frames, so AODV's broadcast route requests are sent at the same rate as data. The channel uses a constant-speed propagation delay, the Friis free-space loss model [33] and a range cutoff at 250 m. Nodes start at random positions and move under the Random Waypoint model: each node picks a random point in the area, moves to it at the configured speed (5 m/s by default), pauses for the configured time (0 s by default) and repeats.

Communication follows a client–server model over UDP. The client on node 0 is connected to the server's address 10.1.1.N, port 9, and sends one 512-byte datagram every $1/\lambda$ seconds (4 packets/s by default) from 2 s to 59 s; the server only receives. Two kinds of packets therefore share the channel: data datagrams on port 9, forwarded by unicast hop by hop, and routing control messages on the protocol's port (AODV 654, OLSR 698, DSDV 269), sent by broadcast or unicast. Every protocol is simulated on the same topologies: run k uses the same initial positions and move-

ment for AODV, OLSR and DSDV. Because positions are random, the source and destination are sometimes within direct range and sometimes several hops apart or disconnected.

7.3 Mathematical Model

At any time t, the network is represented as a graph

$$G(t) = (V, E(t)), \quad E(t) = \{(u, v) : d_{uv}(t) \le R\},$$

where V is the set of N nodes, $d_{uv}(t)$ is the distance between nodes u and v, and $R = 250$ m. A packet can be delivered only while a path from the source to the destination exists in $G(t)$ and AODV has discovered it. Let N_{tx} and N_{rx} be the numbers of data packets sent and received, taken from FlowMonitor for the data flow only. The packet delivery ratio, the loss and the throughput over the traffic period $T = 57$ s, with B_{rx} bytes received, are

$$\text{PDR}(\%) = \frac{N_{\text{rx}}}{N_{\text{tx}}} \times 100,$$

$$N_{\text{loss}} = N_{\text{tx}} - N_{\text{rx}}, \qquad \text{Loss}(\%) = \frac{N_{\text{loss}}}{N_{\text{tx}}} \times 100 = 100 - \text{PDR},$$

$$\text{Throughput(kbps)} = \frac{B_{\text{rx}} \times 8}{T \times 1000}.$$

Throughput is measured at the IP layer, so each 512-byte payload counts as 540 bytes (with 8 bytes of UDP and 20 bytes of IP header); at 100% delivery and 4 packets/s this gives $228 \times 540 \times 8 / 57 = 17.28$ kbps. The average end-to-end delay of the received packets is

$$\overline{D} = \frac{1}{N_{\text{rx}}} \sum_{i=1}^{N_{\text{rx}}} (t_i^{\text{rx}} - t_i^{\text{tx}}),$$

which includes waiting for route discovery, queuing, MAC retransmissions and transmission on every hop. With D_i the delay of the i-th received packet, the average jitter, and the normalised routing load with C routing control packets transmitted by all nodes, are

$$\bar{J} = \frac{1}{N_{\text{rx}} - 1} \sum_{i=2}^{N_{\text{rx}}} |D_i - D_{i-1}|, \qquad \text{NRL} = \frac{C}{N_{\text{rx}}}.$$

A scenario is described by $\theta = (N, v, \lambda)$, where v is the node speed and λ the packet rate. For a metric m measured in runs $k = 1, \ldots, K$ ($K = 10$), the mean, the sample standard deviation and the 95% confidence interval are

$$\overline{m}(\theta) = \frac{1}{K} \sum_{k=1}^{K} m(\theta, k),$$

$$s(\theta) = \sqrt{\frac{1}{K-1} \sum_{k=1}^{K} \big(m(\theta, k) - \overline{m}(\theta)\big)^2},$$

$$\text{CI}_{95} = \overline{m}(\theta) \pm t_{0.975,\,K-1} \frac{s(\theta)}{\sqrt{K}},$$

where $t_{0.975,\,9} = 2.262$ is the Student t quantile for ten runs.

7.4 Proposed Algorithm

The protocol under study is AODV. Figure 7.2 shows how it finds a route when node S has data for node D and no route exists: the route request floods outwards and the reply returns only along the reverse path that the request created. Listing 7.1 gives AODV's route discovery and maintenance algorithm, Table 7.1 compares it with the two proactive baselines, and Listing 7.2 gives the experiment procedure.

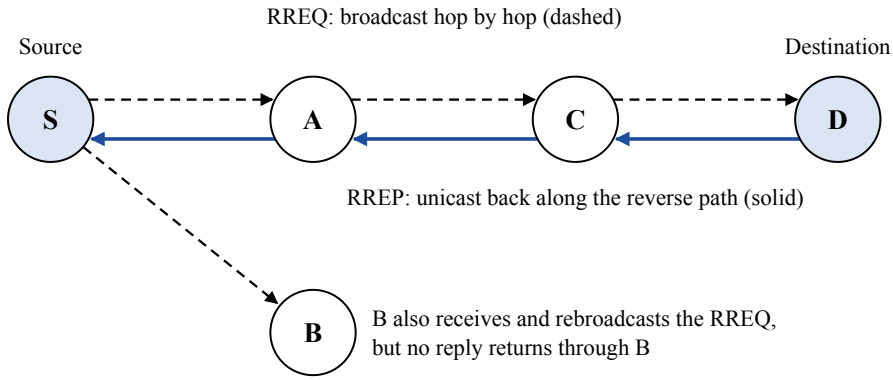


Figure 7.2: AODV route discovery from source S to destination D.

Table 7.1: Routing protocols compared (ns-3 default settings) [4, 8, 9]

Protocol	Type	UDP port	How a route is obtained	Control messages
AODV	Reactive, distance vector	654	Discovered on demand; packets wait in a buffer meanwhile	RREQ, RREP, RERR; Hello every 1 s on active routes
OLSR	Proactive, link state	698	Always in the table; packets without a route are dropped	HELLO every 2 s; TC every 5 s, flooded by multipoint relays
DSDV	Proactive, distance vector	269	Always in the table; packets wait in a buffer until a route appears	Full update every 15 s; triggered updates on change

Listing 7.1: AODV route discovery and maintenance at a node (based on RFC 3561)

```
On a data packet for destination D:
 1. If a valid route to D exists, forward the packet to its next hop.
 2. Else buffer the packet and broadcast RREQ(source, seq, D, RREQ ID).
On receiving RREQ(source, RREQ ID) from neighbour n:
 3. If this (source, RREQ ID) was seen before, discard it.
 4. Install or refresh the reverse route to the source via n.
 5. If this node is D, or has a route to D that is fresh enough,
    unicast an RREP towards the source; else rebroadcast the RREQ.
On receiving RREP for D from neighbour n:
 6. Install or refresh the forward route to D via n.
 7. If this node is the source, send the buffered packets;
    else forward the RREP along the reverse route.
On detecting a broken link to next hop h:
 8. Invalidate every route through h and send RERR to its users.
 9. A source that still has data for D restarts discovery (step 2).
```

Listing 7.2: Experiment procedure

```
Input:  experiments A (nodes = 5, 10, 15, 20), B (speed = 1, 5, 10, 15 m/s),
        C (packetRate = 10 .. 800 packets/s); protocols P = {AODV, OLSR, DSDV};
        runs K = 10
Output: one CSV row per run; mean, SD, 95% CI and paired tests; graphs
1. Build the manet-routing program with ns-3 (only if the code changed).
2. For each experiment E, protocol p and value x of the varied parameter:
     For run k = 1 .. K:
       a. Set the parameter to x and keep the others at their defaults.
       b. Set random seed 1 and run number k (same topology for every p).
       c. Create nodes, Wi-Fi, mobility, IPv4 + protocol p and the CBR flow.
       d. Run 60 s with FlowMonitor and the packet counters installed.
       e. Compute Tx, Rx, loss, PDR, throughput, delay, jitter, overhead.
       f. Append one row to the experiment's CSV file.
3. Group the rows by protocol and parameter value; compute mean, SD and
   95% CI; compare protocols with paired t-tests (paired by run number).
4. Plot every metric against the varied parameter, one line per protocol.
```

7.5 Socket Programming Implementation

Socket programming appears in two places. The CBR client and server are ns-3 applications (`UdpClient` and `UdpServer`) that communicate through UDP sockets, and the routing agent on every node (AODV, OLSR or DSDV) sends and receives its control messages through its own UDP sockets. ns-3 has no operating-system kernel, so these are C++ method calls on simulated sockets rather than real system calls, but the API deliberately mirrors BSD sockets [7, 12]. Table 7.2 maps each call to its BSD equivalent and its use in this project, and Listing 7.3 shows the calls in the ns-3.48 source code for the client, the server and AODV; OLSR and DSDV use the same calls on their own ports.

The main difference from a BSD program is how data is received. A BSD server usually blocks in `recvfrom()` until a datagram arrives. In a discrete-event simulator nothing may block, so the server registers a receive callback with `SetRecvCallback()`; the simulator calls it when a packet arrives, and the callback reads all waiting packets with `RecvFrom()`. The project's program creates these sockets through the helper classes `UdpClientHelper`, `UdpServerHelper` and the routing helpers (Listing 7.4).

Table 7.2: BSD socket calls and their ns-3 equivalents in this project

BSD socket call	ns-3 Socket API	Use in this project
`socket()`	`Socket::CreateSocket(node, UdpSocketFactory)`	Client, server and AODV sockets
`bind()`	`Bind(InetSocketAddress(addr, port))`	Server: port 9; routing: 654 / 698 / 269; client: any free port
`connect()`	`Connect(InetSocketAddress(dst, 9))`	Client fixes the server address 10.1.1.*N*:9
`send()`	`Send(packet)`	Client sends one datagram every 1/λ s
`sendto()`	`SendTo(packet, 0, InetSocketAddress(dst, 654))`	Routing protocols send control messages
`recvfrom()`	`RecvFrom(from)`	Server counts data; routing reads control messages
blocking read / `select()`	`SetRecvCallback(callback)`	Event-driven reception at server and routing
`setsockopt(SO_BROADCAST)`	`SetAllowBroadcast(true)`	RREQ, HELLO and update broadcasts
`setsockopt(SO_BINDTODEVICE)`	`BindToNetDevice(device)`	One routing socket per Wi-Fi interface
`close()`	`Close()`	When an application or interface stops

Listing 7.3: Socket calls of the client, the server and AODV in ns-3.48 (simplified)

```
// CBR client on node 0 (SourceApplication / UdpClient)
m_socket = Socket::CreateSocket(GetNode(), m_protocolTid); // UDP
m_socket->Bind();                     // any free local port
m_socket->Connect(m_peer);            // server 10.1.1.N, port 9
m_socket->Send(p);                    // one datagram every 1/λ s

// CBR server on node N-1 (SinkApplication / UdpServer)
m_socket->Bind(local);                // any address, port 9
m_socket->SetRecvCallback(MakeCallback(&UdpServer::HandleRead, this));
while (auto packet = socket->RecvFrom(from)) { ... }  // count packet

// AODV routing agent on every node (aodv-routing-protocol.cc)
socket = Socket::CreateSocket(GetObject<Node>(),
                              UdpSocketFactory::GetTypeId());
socket->SetRecvCallback(MakeCallback(&RoutingProtocol::RecvAodv, this));
socket->BindToNetDevice(l3->GetNetDevice(i));
socket->Bind(InetSocketAddress(iface.GetLocal(), AODV_PORT)); // 654
socket->SetAllowBroadcast(true);
socket->SendTo(packet, 0, InetSocketAddress(destination, AODV_PORT));
```

7.6 Implementation of the Simulation Program

The simulation is a single C++ program, `manet-routing.cc` (about 450 lines), whose parameters, including `--protocol`, are set on the command line. It extends the original `manet-aodv.cc` without changing how AODV is simulated, so its AODV results are identical run for run. Two counters were added (Listing 7.4). First, routing overhead is counted with the `Ipv4L3Protocol` “Tx” trace, which

reports every IPv4 packet a node transmits; a packet counts as routing control when its UDP port is the protocol's port. FlowMonitor cannot be used here because it ignores broadcast packets. Second, sent data packets are counted with the client's "Tx" trace, because OLSR drops a packet at the source when it has no route, before FlowMonitor sees it.

Listing 7.4: Protocol selection and the two packet counters (excerpt of `manet-routing.cc`)

```
static void CountRoutingTx(Ptr<const Packet> packet, Ptr<Ipv4>, uint32_t)
{
    Ptr<Packet> copy = packet->Copy();
    Ipv4Header ipHeader;   copy->RemoveHeader(ipHeader);
    if (ipHeader.GetProtocol() != 17) return;                  // UDP only
    UdpHeader udpHeader;   copy->PeekHeader(udpHeader);
    if (udpHeader.GetDestinationPort() == g_routingPort ||
        udpHeader.GetSourcePort() == g_routingPort) {          // 654, 698 or 269
        g_ctrlPackets++;   g_ctrlBytes += packet->GetSize();
    }
}

if (protocol == "AODV")      internet.SetRoutingHelper(aodv);
else if (protocol == "OLSR") internet.SetRoutingHelper(olsr);
else                         internet.SetRoutingHelper(dsdv);
internet.Install(nodes);
Config::ConnectWithoutContext("/NodeList/*/$ns3::Ipv4L3Protocol/Tx",
                              MakeCallback(&CountRoutingTx));
clientApp.Get(0)->TraceConnectWithoutContext("Tx", MakeCallback(&CountAppTx));
```

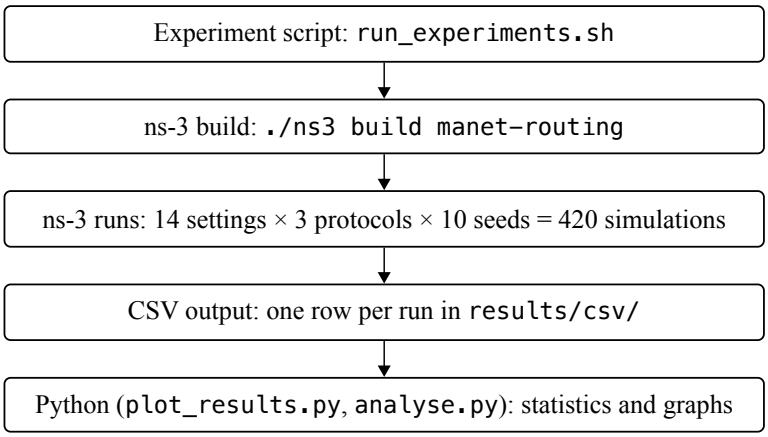


Figure 7.3: Sequence of operations from simulation to graphs.

8. Performance Evaluation

8.1 Experimental Setup

All simulations ran on macOS 15.6.1 on Apple Silicon (M2), with ns-3.48 built by Apple clang 16.0.0, CMake 4.4.3 and Ninja 1.13.2, and with Python 3.13.7, pandas, SciPy and matplotlib for processing. Table 8.1 lists the default simulation parameters, and Table 8.2 the three experiments, each of which changes one parameter while the others keep their defaults.

Table 8.1: Simulation parameters (defaults)

Parameter	Value
Number of nodes; area	10; 700 m × 700 m
Wi-Fi standard and mode	IEEE 802.11b, ad-hoc, 11 Mbps (DSSS)
Propagation	Constant-speed delay; Friis loss; 250 m range cutoff
Mobility model	Random Waypoint, 5 m/s, 0 s pause
Network layer and routing	IPv4; AODV, OLSR or DSDV (ns-3 defaults)
Traffic	UDP client–server, CBR, 512 bytes, 4 packets/s
Simulation time	60 s (traffic from 2 s to 59 s)
Repetitions	10 runs per setting and protocol (seed 1, run 1–10)

Table 8.2: Experiments and parameter values

Experiment	Parameter varied	Values	Kept fixed
A	Number of nodes	5, 10, 15, 20	5 m/s, 4 packets/s
B	Node speed	1, 5, 10, 15 m/s	10 nodes, 4 packets/s
C	Traffic load (CBR rate)	10, 50, 100, 200, 400, 800 packets/s	10 nodes, 5 m/s

Each value is simulated for each protocol with run numbers 1 to 10, each giving a different random placement and movement of the nodes (14 settings × 3 protocols × 10 runs = 420 runs). Fixed random streams make every run exactly reproducible and give the three protocols identical topologies.

8.2 Performance Metrics

- **Packet delivery ratio (PDR)** and **throughput**: delivered share and rate of the data (higher is better).
- **Average delay** and **jitter**: transit time of delivered packets and its variation (lower is better).
- **Packet loss**, including packets dropped at the source for lack of a route (lower is better).
- **Routing overhead**: control packets sent by all nodes, and the NRL (lower is better).

The metrics are defined in Section 7.3; delay and jitter cover received packets only. For every setting the mean, sample standard deviation (SD) and 95% confidence interval are reported. AODV is compared with OLSR and DSDV by paired *t*-tests on runs with the same number, which share a topology, checked with Wilcoxon signed-rank tests; $p < 0.05$ counts as significant.

9. Output and Results

9.1 Sample Simulation Output

Each run prints its parameters and metrics. Listing 9.1 shows the default scenario with AODV and run number 2, in which the source and destination communicate over multi-hop routes that break while nodes move.

Listing 9.1: Output of `./scripts/run_single.sh --protocol=AODV --run=2`

```
============================================================
 AODV MANET Simulation Results (ns-3.48)
============================================================
 Parameters
   Routing protocol    : AODV
   Nodes               : 10
   Source -> Dest      : node 0 (10.1.1.1) -> node 9 (10.1.1.10)
 Metrics
   Transmitted packets : 228
   Received packets    : 208
   Lost packets        : 20 (8.772 %)
   Dropped at source   : 0 (no route)
   Packet Delivery Ratio : 91.228 %
   Throughput          : 15.764 kbps
   Average E2E delay   : 226.380 ms
   Average jitter      : 61.703 ms
 Routing overhead
   Control packets     : 705 (33986 bytes)
   Normalised load     : 3.389 control packets per delivered packet
============================================================
```

On the same topology, OLSR delivered 159 packets (69.7%) with a 2.4 ms delay but dropped 41 at the source for lack of a route, while DSDV, which buffers like AODV, delivered 179 (78.5%) with a 194 ms delay.

9.2 Observations

9.2.1 Experiment A: Number of Nodes

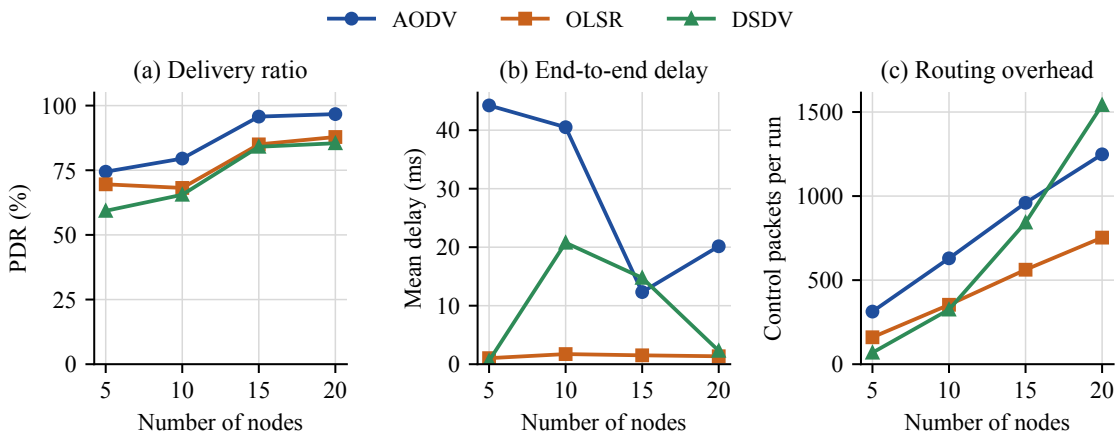


Figure 9.1: Delivery ratio, delay and routing overhead against the number of nodes.

Table 9.1: Effect of the number of nodes (mean of 10 runs, PDR as mean ± SD; 5 m/s, 4 packets/s)

Nodes	Protocol	PDR (%)	Throughput (kbps)	Delay (ms)	Jitter (ms)	Control pkts	NRL
5	AODV	74.4 ± 34.9	12.9	44.2	15.5	313	3.79
	OLSR	69.6 ± 32.7	12.0	1.0	0.2	160	1.91
	DSDV	59.3 ± 33.9	10.2	0.6	0.0	68	0.36
10	AODV	79.5 ± 30.2	13.7	40.5	13.8	630	5.53
	OLSR	68.2 ± 31.7	11.8	1.7	0.5	353	3.14
	DSDV	65.5 ± 36.2	11.3	20.8	4.5	325	4.19
15	AODV	95.7 ± 7.7	16.5	12.3	10.2	960	4.42
	OLSR	85.0 ± 8.5	14.7	1.5	0.5	562	2.94
	DSDV	84.1 ± 17.3	14.5	14.8	5.4	843	4.62
20	AODV	96.7 ± 4.2	16.7	20.2	11.1	1,248	5.68
	OLSR	87.9 ± 14.4	15.2	1.4	0.4	753	3.89
	DSDV	85.4 ± 17.4	14.8	2.3	1.2	1,542	8.31

All three protocols deliver more as density rises, because partitions become rare: AODV from 74.4% to 96.7%, OLSR from 69.6% to 87.9% and DSDV from 59.3% to 85.4%. AODV stays ahead at every density. OLSR's delay stays near 1–2 ms because it drops packets that have no route instead of holding them, whereas AODV (12–44 ms) and DSDV (up to 21 ms) buffer packets until a route appears, which costs delay but saves packets. Routing overhead grows with the number of nodes for every protocol; OLSR's grows most slowly, and DSDV's grows fastest and exceeds AODV's at 20 nodes (1,542 against 1,248 control packets per run).

9.2.2 Experiment B: Node Speed

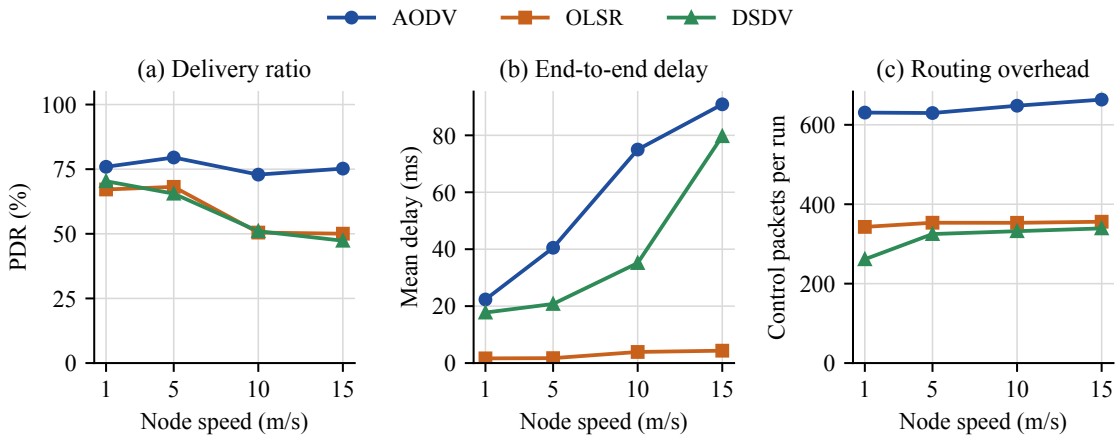


Figure 9.2: Delivery ratio, delay and routing overhead against node speed.

Table 9.2: Effect of node speed (mean of 10 runs, PDR as mean ± SD; 10 nodes, 4 packets/s)

Speed (m/s)	Protocol	PDR (%)	Throughput (kbps)	Delay (ms)	Jitter (ms)	Control pkts	NRL
1	AODV	75.9 ± 34.8	13.1	22.3	8.3	631	3.28
	OLSR	67.1 ± 35.4	11.6	1.6	0.1	343	2.19
	DSDV	70.4 ± 33.7	12.2	17.7	10.6	261	1.51
5	AODV	79.5 ± 30.2	13.7	40.5	13.8	630	5.53
	OLSR	68.2 ± 31.7	11.8	1.7	0.5	353	3.14
	DSDV	65.5 ± 36.2	11.3	20.8	4.5	325	4.19
10	AODV	72.9 ± 28.0	12.6	75.0	27.3	648	10.18
	OLSR	50.5 ± 32.9	8.7	3.9	3.0	353	6.78
	DSDV	51.0 ± 32.1	8.8	35.1	8.0	332	6.26
15	AODV	75.2 ± 24.0	13.0	90.9	21.2	664	4.59
	OLSR	50.0 ± 22.4	8.6	4.3	2.9	356	3.75
	DSDV	47.3 ± 20.1	8.2	79.7	17.5	339	3.64

Speed separates the protocols most clearly. AODV keeps its PDR between 72.9% and 79.5% at every speed because it repairs broken routes on demand. OLSR and DSDV fall from 67.1% and 70.4% at 1 m/s to 50.0% and 47.3% at 15 m/s: their tables are refreshed only every 2–5 s (OLSR) or 15 s (DSDV), so at higher speeds they often use routes that have already broken. AODV's price is delay, which rises from 22.3 ms to 90.9 ms as more packets wait for a new route, and overhead: it sends about twice as many control packets as the proactive protocols (630–664 against 261–356 per run).

9.2.3 Experiment C: Traffic Load

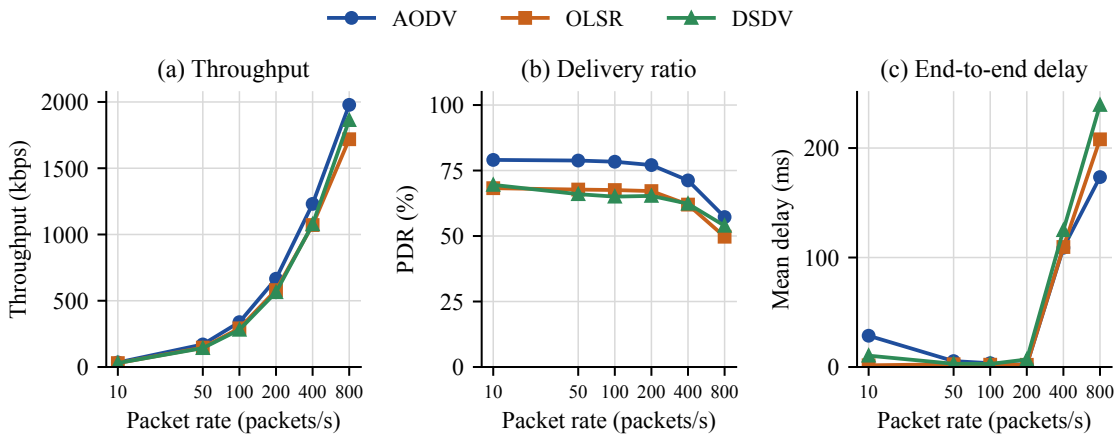


Figure 9.3: Throughput, delivery ratio and delay against packet rate (logarithmic axis).

Table 9.3: Effect of traffic load (mean of 10 runs, PDR as mean ± SD; 10 nodes, 5 m/s)

Rate (pkt/s)	Protocol	PDR (%)	Throughput (kbps)	Delay (ms)	Jitter (ms)	Control pkts	NRL
10	AODV	79.1 ± 30.3	34.2	28.6	6.8	633	2.26
	OLSR	68.3 ± 31.8	29.5	1.6	0.2	353	1.25
	DSDV	69.5 ± 28.9	30.0	10.4	2.0	327	1.04
50	AODV	78.8 ± 29.9	170.2	5.4	0.6	637	0.45
	OLSR	67.7 ± 32.6	146.3	2.5	0.7	353	0.27
	DSDV	66.0 ± 36.4	142.5	2.9	1.1	322	0.33
100	AODV	78.4 ± 29.8	338.5	3.5	0.2	645	0.23
	OLSR	67.5 ± 32.6	291.8	2.1	0.2	352	0.13
	DSDV	65.0 ± 34.5	281.0	2.8	0.9	321	0.15
200	AODV	77.1 ± 29.3	666.0	2.1	0.1	1,081	0.22
	OLSR	67.2 ± 33.0	580.2	2.0	0.1	353	0.07
	DSDV	65.3 ± 34.9	564.4	6.9	0.4	314	0.07
400	AODV	71.2 ± 31.0	1,230.8	108.9	1.0	1,054	0.09
	OLSR	62.0 ± 35.0	1,071.3	109.6	1.6	352	0.04
	DSDV	62.3 ± 33.4	1,077.1	125.0	0.5	311	0.03
800	AODV	57.2 ± 36.2	1,978.3	173.5	2.6	1,634	0.12
	OLSR	49.7 ± 39.5	1,718.8	208.0	1.9	348	0.04
	DSDV	53.9 ± 36.9	1,863.3	239.4	1.6	304	0.03

Up to 200 packets/s all three protocols carry the traffic: throughput grows in proportion to the load and PDR stays at its low-load level (about 77–79% for AODV and 65–70% for OLSR and DSDV). At 400 and 800 packets/s the shared channel congests for every protocol; at 800 packets/s PDR falls to 57.2% (AODV), 53.9% (DSDV) and 49.7% (OLSR) and delays rise to 170–240 ms. AODV's control traffic rises with load (633 to 1,634 packets per run) as congestion breaks links and triggers new discoveries, while OLSR's and DSDV's periodic overhead stays nearly constant.

9.3 Comparative Analysis

Table 9.4 compares AODV with the two proactive baselines over the 13 distinct settings (the baseline setting belongs to both Experiments A and B), using paired *t*-tests on runs that share the same topology.

Table 9.4: AODV compared with OLSR and DSDV over 13 settings (paired t-test, $p < 0.05$)

Measure	AODV vs OLSR	AODV vs DSDV
Mean PDR over all settings	77.9% vs 66.2%	77.9% vs 65.0%
PDR advantage per setting	+4.9 to +25.2 points	+3.3 to +27.9 points
AODV PDR significantly higher	12 of 13 settings	9 of 13 settings
AODV PDR significantly lower	None	None
Control packets	AODV more in 12 of 13	AODV more in 11; DSDV more at 20 nodes
Mean delay	AODV higher on average; significant in 1	Significant only at 800 pkt/s (DSDV higher)

AODV delivered the most data, never significantly less than either baseline, with its largest margin at high speed. Two mechanisms explain this. AODV finds a route when it is needed and repairs it as soon as a link breaks, while OLSR and DSDV rely on periodic updates and keep using stale routes in between. AODV also holds packets during route discovery, whereas OLSR drops a packet at once when it has no route; these source drops make up 72–75% of all OLSR losses. The price is overhead and delay: AODV sent about twice as many control packets, and its buffered packets raise its mean delay and jitter. These results agree with earlier studies, which found that on-demand protocols keep higher delivery under mobility while DSDV degrades [18], and that AODV copes best as load and mobility increase [20].

Repeated runs were essential: single runs ranged from 0% to 100% delivery for every protocol. Because the protocols share the same ten topologies, the comparison is paired, so the protocol differences are significant even though the spread between topologies is large.

9.4 Validation of the Proposed Approach

Table 9.5: Validation checks

Check	Expected	Observed
Regression	AODV equal to the original study	All 140 AODV runs identical
Packets sent per run	57 s × λ (228 at 4 packets/s)	Exact in all 420 runs
Loss and PDR	Loss (%) = 100 − PDR	Exact in all 420 runs
Throughput and load	Offered load × PDR	Within 0.002 kbps (Exp. C)
OLSR overhead	N × 30 HELLOs (every 2 s) + TC	160–753 packets; HELLOs alone 150–600
Source drops	None for buffering protocols	0 in every AODV and DSDV run

All checks in Table 9.5 passed. Within AODV alone, only the load effect is significant ($p = 0.013$ from 200 to 800 packets/s; density $p = 0.075$, speed $p = 0.38$). Wilcoxon tests confirmed the PDR results and every overhead difference.

10. Conclusion

Findings. A MANET was implemented in ns-3.48 in which a UDP client–server application sends CBR traffic over AODV, OLSR or DSDV, and 420 runs were simulated on shared topologies. AODV delivered from 74.4% to 96.7% of packets as the network grew from 5 to 20 nodes, kept its delivery near 75% from 1 to 15 m/s, and reached 666 kbps at 200 packets/s before congestion lowered its PDR to 57.2% at 800 packets/s. Its packet loss fell from 58.3 to 7.5 packets per run as the network grew.

Comparison with the baselines. AODV delivered 77.9% of packets on average against 66.2% for OLSR and 65.0% for DSDV, significantly more in 12 of 13 settings than OLSR and in 9 of 13 than DSDV, and never significantly less. Its advantage was largest under mobility, where OLSR and DSDV fell to 50.0% and 47.3%. The cost was about twice the routing overhead and a higher delay and jitter (40.5 ms and 13.8 ms against 1.7 ms and 0.5 ms for OLSR in the default scenario), since AODV buffers packets during route discovery.

Contribution to the research gap. The project provides a reproducible comparison of standard AODV with proactive baselines on a current ns-3 release, varying density, speed and load one at a time and reporting overhead, spread and statistical confidence, which is the gap selected in Chapter 5.

Limitations. The radio model uses a hard 250 m cutoff, only one CBR flow was simulated, Random Waypoint is synthetic, all protocols use default settings with traffic starting at 2 s, which gives the proactive protocols little time to build their tables, and ten runs are too few for the density and speed effects within AODV to be significant.

Future work. Tune the protocol timers and add a warm-up period, add concurrent flows and TCP traffic, use more runs per setting, replace the range cutoff with a fading propagation model, and measure energy use and resilience to attacks such as the black hole attack.

11. References

[1] W. R. Stevens, B. Fenner, and A. M. Rudoff, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2003.

[2] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Hoboken, NJ, USA: Pearson, 2021.

[3] S. Corson and J. Macker, “Mobile Ad hoc Networking (MANET): Routing Protocol Performance Issues and Evaluation Considerations,” RFC 2501, IETF, Jan. 1999. Available: https://www.rfc-editor.org/rfc/rfc2501.

[4] C. E. Perkins, E. M. Belding-Royer, and S. R. Das, “Ad hoc On-Demand Distance Vector (AODV) Routing,” RFC 3561, IETF, Jul. 2003. Available: https://www.rfc-editor.org/rfc/rfc3561.

[5] The Linux man-pages project, “socket(2), bind(2), connect(2), send(2), recv(2), close(2) and udp(7),” *Linux Programmer’s Manual*. Available: https://man7.org/linux/man-pages/. Accessed Oct. 2026.

[6] J. Postel, “User Datagram Protocol,” RFC 768, IETF, Aug. 1980. Available: https://www.rfc-editor.org/rfc/rfc768.

[7] ns-3 Project, “Sockets APIs,” *ns-3 Model Library*, online documentation. Available: https://www.nsnam.org/docs/models/html/sockets-api.html. Accessed Oct. 2026.

[8] C. E. Perkins and P. Bhagwat, “Highly Dynamic Destination-Sequenced Distance-Vector Routing (DSDV) for Mobile Computers,” in *Proc. ACM SIGCOMM*, 1994, pp. 234–244.

[9] T. Clausen and P. Jacquet, “Optimized Link State Routing Protocol (OLSR),” RFC 3626, IETF, Oct. 2003. Available: https://www.rfc-editor.org/rfc/rfc3626.

[10] D. B. Johnson and D. A. Maltz, “Dynamic Source Routing in Ad Hoc Wireless Networks,” in *Mobile Computing*, T. Imielinski and H. F. Korth, Eds. Boston, MA, USA: Kluwer Academic, 1996, pp. 153–181.

[11] C. E. Perkins and E. M. Royer, “Ad-hoc On-Demand Distance Vector Routing,” in *Proc. 2nd IEEE Workshop on Mobile Computing Systems and Applications (WMCSA)*, 1999, pp. 90–100.

[12] G. F. Riley and T. R. Henderson, “The ns-3 Network Simulator,” in *Modeling and Tools for Network Simulation*, K. Wehrle, M. Güneş, and J. Gross, Eds. Berlin, Germany: Springer, 2010, pp. 15–34.

[13] ns-3 Project, “ns-3 Manual and Model Library (AODV, Wi-Fi, Mobility and FlowMonitor),” online documentation. Available: https://www.nsnam.org/documentation/. Accessed Oct. 2026.

[14] ns-3 Project, “ns-3 Tutorial,” online tutorial. Available: https://www.nsnam.org/docs/tutorial/html/. Accessed Oct. 2026.

[15] G. Carneiro, P. Fortuna, and M. Ricardo, “FlowMonitor: A Network Monitoring Framework for the Network Simulator 3 (NS-3),” in *Proc. 4th Int. ICST Conf. Performance Evaluation Methodologies and Tools (VALUETOOLS)*, 2009.

[16] T. Agarwal, V. Buyanni, Y. T. Paul, and Vijeth MD, “manet-routing-comparison: AODV, OLSR and DSDV in a MANET Using ns-3,” source code and results. Available: https://github.com/vijethmd/manet-routing-comparison.

[17] M. Abolhasan, T. Wysocki, and E. Dutkiewicz, “A Review of Routing Protocols for Mobile Ad Hoc Networks,” *Ad Hoc Networks*, vol. 2, no. 1, pp. 1–22, 2004.

[18] J. Broch, D. A. Maltz, D. B. Johnson, Y.-C. Hu, and J. Jetcheva, “A Performance Comparison of Multi-Hop Wireless Ad Hoc Network Routing Protocols,” in *Proc. ACM MobiCom*, 1998, pp. 85–97.

[19] P. Johansson, T. Larsson, N. Hedman, B. Mielczarek, and M. Degermark, “Scenario-based Performance Analysis of Routing Protocols for Mobile Ad-hoc Networks,” in *Proc. ACM MobiCom*, 1999, pp. 195–206.

[20] C. E. Perkins, E. M. Royer, S. R. Das, and M. K. Marina, “Performance Comparison of Two On-Demand Routing Protocols for Ad Hoc Networks,” *IEEE Personal Communications*, vol. 8, no. 1, pp. 16–28, Feb. 2001, doi: 10.1109/98.904895.

[21] M. K. Marina and S. R. Das, “On-demand Multipath Distance Vector Routing in Ad Hoc Networks,” in *Proc. IEEE Int. Conf. Network Protocols (ICNP)*, 2001, pp. 14–23.

[22] T. Camp, J. Boleng, and V. Davies, “A Survey of Mobility Models for Ad Hoc Network Research,” *Wireless Communications and Mobile Computing*, vol. 2, no. 5, pp. 483–502, 2002.

[23] J. Yoon, M. Liu, and B. Noble, “Random Waypoint Considered Harmful,” in *Proc. IEEE INFOCOM*, 2003.

[24] D. Kotz, C. Newport, R. S. Gray, J. Liu, Y. Yuan, and C. Elliott, “Experimental Evaluation of Wireless Simulation Assumptions,” in *Proc. ACM/IEEE Int. Symp. Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*, 2004, pp. 78–82, doi: 10.1145/1023663.1023679.

[25] S. Kurkowski, T. Camp, and M. Colagrosso, “MANET Simulation Studies: The Incredibles,” *ACM SIGMOBILE Mobile Computing and Communications Review*, vol. 9, no. 4, pp. 50–61, 2005.

[26] T. R. Andel and A. Yasinsac, “On the Credibility of MANET Simulations,” *IEEE Computer*, vol. 39, no. 7, pp. 48–54, Jul. 2006.

[27] Z. Mammeri, “Reinforcement Learning Based Routing in Networks: Review and Classification of Approaches,” *IEEE Access*, vol. 7, pp. 55916–55950, 2019, doi: 10.1109/ACCESS.2019.2913776.

[28] I. Alameri, J. Komarkova, T. Al-Hadhrami, and A. Lotfi, “Systematic Review on Modification to the Ad-hoc On-Demand Distance Vector Routing Discovery Mechanics,” *PeerJ Computer Science*, vol. 8, art. e1079, 2022, doi: 10.7717/peerj-cs.1079.

[29] V. K. Quy, V. H. Nam, D. M. Linh, and L. A. Ngoc, “Routing Algorithms for MANET-IoT Networks: A Comprehensive Survey,” *Wireless Personal Communications*, vol. 125, no. 4, 2022, doi: 10.1007/s11277-022-09722-x.

[30] C. Santana, M. Macedo, E. Alves, M. T. Guerreiro, H. V. Siqueira, A. Gokhale, and C. Bastos-Filho, “Bio-Inspired Multi-Objective Algorithms Applied on the Optimization of the AODV’s Routing Recovery Mechanism,” *IEEE Access*, vol. 11, pp. 116480–116496, 2023, doi: 10.1109/ACCESS.2023.3322691.

[31] P. M. Njoroge, J. G. Ndia, and P. M. Mwangi, “A Systematic Review of Ad Hoc On-Demand Distance Vector Routing Protocol for Performance Improvement in Mobile Ad Hoc Networks,” *International Journal of Wireless & Mobile Networks*, vol. 17, no. 2, Apr. 2025, doi: 10.5121/ijwmn.2025.17201.

[32] IEEE Std 802.11b-1999, “Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications: Higher-Speed Physical Layer Extension in the 2.4 GHz Band,” IEEE, 1999.

[33] H. T. Friis, “A Note on a Simple Transmission Formula,” *Proceedings of the IRE*, vol. 34, no. 5, pp. 254–256, 1946.